

因此,如果虚拟机需要访问相关寄存器(例如读取 ICC_IAR1_EL1 寄存器以获得当前中断号),将触发异常并进入虚拟机监控器,虚拟机监控器之后模拟寄存器的访问过程(如将读取结果写入目标寄存器)。另一方面,在插入虚拟中断时,虚拟机监控器直接调用虚拟机内的中断处理函数。之后,虚拟机读取 ICC_IAR1_EL1 寄存器获取中断号,并处理中断。通过软件模拟的方法实现中断会带来较多的虚拟机下陷,因而性能较差。

为了优化中断虚拟化的性能,ARM GIC (Generic Interrupt Controller) 在第二版中增加了对中断虚拟化的硬件支持。自此之后,虚拟机监控器可以为虚拟机配置硬件实现的虚拟寄存器,虚拟机可使用这些寄存器控制中断的行为(例如开关中断)。虚拟机在访问这些寄存器时不会产生任何虚拟机下陷。此外,插入中断的方式也得到了优化。虚拟机监控器可以使用 GIC 提供的接口插入中断,此中断将在 CPU 恢复虚拟机执行时由硬件自动插入并直接调用对应的中断处理函数。

硬件虚拟化中的虚拟中断插入过程

当物理设备产生物理中断并发送到某一个物理核时,如果此时虚拟机正在运行,将会引起该虚拟机下陷。在处理完此物理中断后,虚拟机监控器判断出该物理中断相关的 I/O 数据需要传输给某虚拟机。因此,它先将数据拷贝进虚拟机内存区域,并向此虚拟机插入虚拟中断。为了插入虚拟中断,虚拟机监控器在 GICH_LR<n>_EL2 寄存器中写入该虚拟中断的信息,例如中断号和优先级等。之后,虚拟机监控器恢复 vCPU 的执行,在这一时刻,硬件自动根据 GICH_LR<n>_EL2 中的信息向 vCPU 插入一个中断,并调用其中断处理函数。在虚拟机处理该中断的过程中,写入 GICC_EOIR1_EL1 等寄存器时不会触发任何虚拟机下陷。

上述中断虚拟化方法虽然已经大大提升了虚拟机在处理中断时的性能,但每次触发物理中断时依然会引起虚拟机下陷。尤其是当设备已经直通给虚拟机时,由物理中断产生的下陷会对虚拟机的 I/O 性能产生较大的影响。为了解决该问题,不同体系结构推出了新的中断虚拟化技术,允许物理设备直接向虚拟机发送物理中断(不产生虚拟机下陷)。这种技术在 Intel 体系结构下被称为 Posted Interrupt,它的转发机制主要由 IOMMU 实现。在 AArch64 体系结构下,这种中断由 GIC 的 ITS (Interrupt Translation Service) 实现。

在调度虚拟机之前,虚拟机监控器配置 ITS 中的中断翻译表 (Interrupt Translation Table, ITT) 等信息,将直通设备的物理中断映射成某个虚拟中断。直通设备发送物理中断后, GIC 查询中断翻译表等信息,并将物理中断的信息翻译成对应的虚拟中断。之后, GIC 直接将该虚拟中断插入对应的虚拟机,并调用客户操作系统注册的中断处理函数。处理此中断的过程与非虚拟化场景下完全一致,不会产生任何虚拟机下陷。因此,使用该技术之后,物理中断的产生、翻译、插入和处理过程不需要虚拟机监控器的介入,完全由设备和 GIC ITS 配合完成,可进一步提升中断虚拟化的性能。

14.7 案例分析：QEMU/KVM

本节主要知识点

- KVM 的接口是什么？如何在用户态使用这些接口？
- KVM 和 QEMU 如何协同工作？
- KVM 的运行机制是什么？

KVM (Kernel-based Virtual Machine) 是运行在 Linux 内核中的硬件虚拟化管理模块，用户态的程序可通过 KVM 暴露的用户态接口使用硬件虚拟化的功能。图 14.26 展示了 KVM 与用户态程序以及虚拟机的关系。首先，在用户态程序运行之前，KVM 通过 Linux 在 /dev 目录下注册并初始化 kvm 设备文件。之后，运行在 EL0 的用户态程序可通过 `ioctl` 等系统调用与 /dev/kvm 设备文件交互，以调用内核态 KVM 的 API。KVM 收到 API 调用后，在 Linux 内核中创建相应的虚拟机、vCPU、第二阶段页表等数据结构。如图 14.26 所示，此用户态程序调用了 KVM 的 `KVM_SET_USER_MEMORY_REGION` API 后，KVM 据此建立该虚拟机的第二阶段页表。在此程序调用 `KVM_RUN` API 时，KVM 使用 `eret` 指令进入某 vCPU 相关的代码位置开始执行。

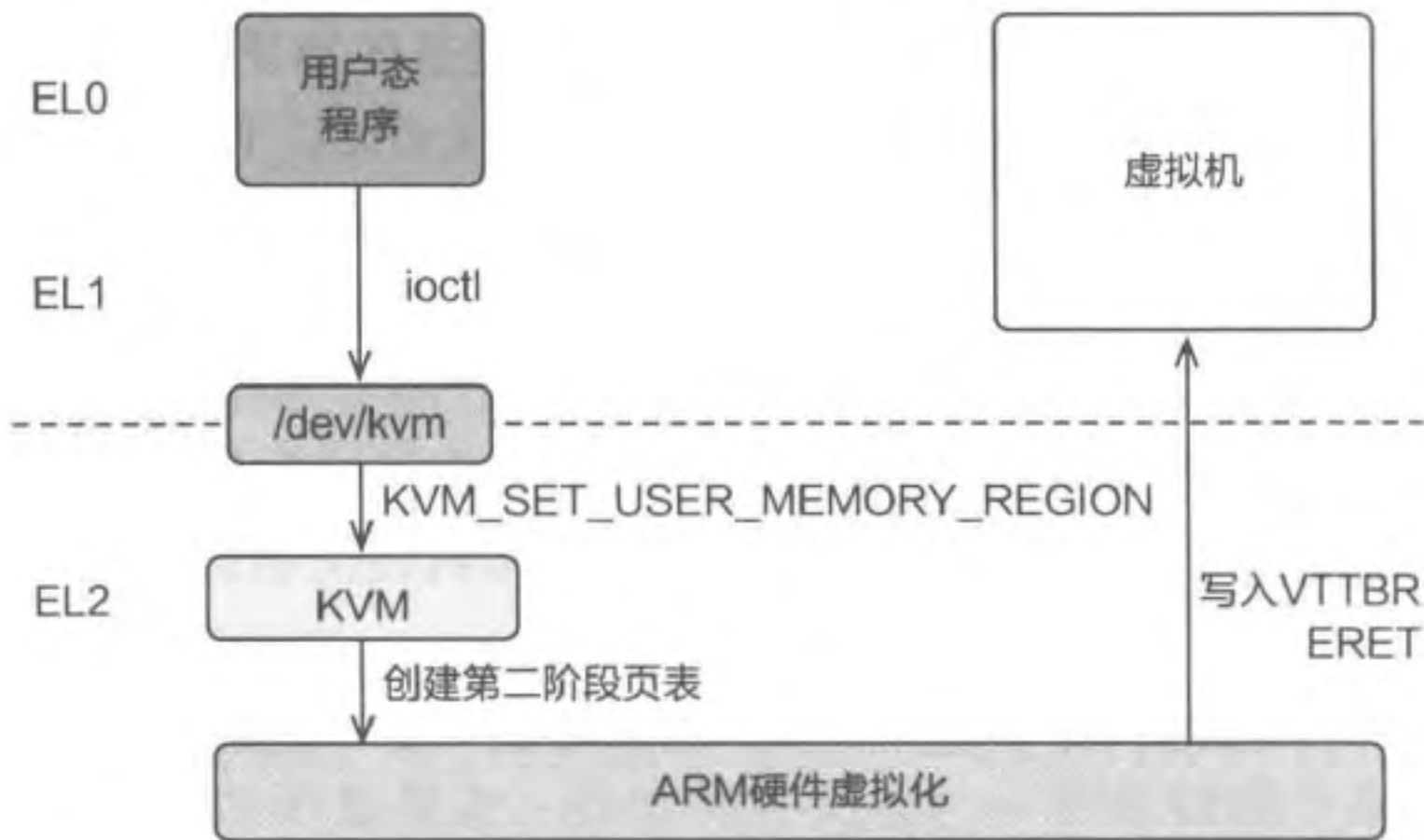


图 14.26 用户态程序使用 KVM 接口运行虚拟机

14.7.1 KVM API 和一个简单的虚拟机监控器

KVM 向用户态提供了 100 多个不同的 API，以使用户态的程序对虚拟机进行管理 (表 14.6 展示了其中 7 个重要的 API)。通过这些 API，用户态程序可以完成创建虚拟机、创建虚拟机内的 vCPU、设置 vCPU 的寄存器状态等操作。读者可能会认为写一个虚拟机监控器非常复杂，实际上，通过 KVM 的 API，使用短短几十行代码，我们就可以创建一个非常简单的虚拟机监控器。

表 14.6 部分 KVM API

API	解 释
KVM_GET_API_VERSION	查看 KVM API 的版本号
KVM_CREATE_VM	在 KVM 内创建一个空的虚拟机，该虚拟机没有 vCPU
KVM_CREATE_VCPU	在某虚拟机内创建一个 vCPU
KVM_GET_ONE_REG	读取某个 vCPU 中特定寄存器的值
KVM_SET_ONE_REG	设置某个 vCPU 中特定寄存器的值
KVM_SET_USER_MEMORY_REGION	设置虚拟机的一块客户物理地址区域
KVM_RUN	运行某个 vCPU

代码片段 14.2 展示了一段简化的虚拟机监控器和虚拟机的代码。这段代码的逻辑主要分成以下五步（注意，此处虚拟机监控器没有打开虚拟机内的页表功能，所以使用了客户物理地址）：

- 第一步：使用 open 系统调用打开 /dev/kvm 设备文件，并得到此文件的文件描述符。
- 第二步：创建虚拟机，并将 vm_code 中保存的虚拟机二进制代码映射到 0x6000 的客户物理地址。
- 第三步：创建 vCPU，并不断调用 KVM_SET_ONE_REG 依次初始化通用寄存器的值（例如 X0）。其中最重要的是将 PC 寄存器的值设置为 0x6000，也就是即将执行二进制代码的起始位置。
- 第四步：使用 KVM_RUN API 执行虚拟机的代码。
- 第五步：处理虚拟机下陷，处理完成后，通过 KVM_RUN 从中断处继续执行虚拟机。

代码片段 14.2 使用 KVM API 实现一个简单的运行在用户态的 Type-2 虚拟机监控器
(这段代码省略了一些变量声明与系统调用错误处理的过程)

```
1 int main(void)
2 {
3     const uint8_t vm_code[] = {
4         //virtual machine binary code
5     };
6     struct kvm_run *run;
7     //其他变量声明省略
8
9     kvm = open("/dev/kvm", O_RDWR | O_CLOEXEC);
10    vmfd = ioctl(kvm, KVM_CREATE_VM, (unsigned long)0);
11
12    // 分配一个页对齐的内存位置，用来存储代码
13    mem = mmap(NULL, 0x1000, PROT_READ | PROT_WRITE,
14               MAP_SHARED | MAP_ANONYMOUS, -1, 0);
15    memcpy(mem, vm_code, sizeof(vm_code));
```

```

16
17 // 将此代码页映射到 0x6000 的客户物理地址
18 struct kvm_userspace_memory_region region = {
19     .slot = 0,
20     .guest_phys_addr = 0x6000,
21     .memory_size = 0x1000,
22     .userspace_addr = (uint64_t)mem,
23 };
24 ret = ioctl(vmfd, KVM_SET_USER_MEMORY_REGION,
25             &region);
26
27 vcpufd = ioctl(vmfd, KVM_CREATE_VCPU,
28               (unsigned long)0);
29
30 // 将内核中的 kvm_run 结构映射到用户态
31 ret = ioctl(kvm, KVM_GET_VCPU_MMAP_SIZE, NULL);
32 mmap_size = ret;
33 run = mmap(NULL, mmap_size, PROT_READ | PROT_WRITE,
34            MAP_SHARED, vcpufd, 0);
35
36 struct kvm_one_reg reg;
37 // 初始化 reg struct, 将 0x6000 的值写入 PC 寄存器中
38 // 也可以通过此过程初始化其他寄存器, 例如 X0 等
39 ret = ioctl(vcpufd, KVM_SET_ONE_REG, &reg);
40
41 // 不断执行虚拟机的代码, 并处理相关的 VM exits
42 while (1) {
43     ret = ioctl(vcpufd, KVM_RUN, NULL);
44     switch (run->exit_reason) {
45         // 处理虚拟机下陷
46     }
47 }
48
49 ...
50 }

```

14.7.2 KVM 和 QEMU

从上面的例子可以看出, 我们可将 KVM 理解为硬件虚拟化机制的驱动程序。然而, 如何使用这个驱动程序, 则需要由用户态的应用程序决定。也就是说, KVM 决定使用硬件虚拟化的“机制”, 而用户态程序提供硬件虚拟化的使用“策略”。

小思考

KVM 和 QEMU 这种机制和策略分离的设计思想有什么好处吗?

一方面, 可针对不同场景设计和实现专用的用户态虚拟机监控器, 无须修改提供

机制的 KVM。例如，亚马逊公司为无服务计算场景设计了 FireCracker，可完全替代 QEMU，但它依然使用 KVM 提供的硬件虚拟化机制。另一方面，通过分离机制与策略，可降低运行在最高特权级的 KVM 代码复杂度，而将复杂的策略逻辑运行在较低特权级，从而在一定程度上提升系统的可靠性。

为了支持功能完整的虚拟机，代码片段 14.2 中的简单虚拟机监控器是远远不够的。它不仅无法提供对 I/O 虚拟化的支持，也不支持多 vCPU。为此，我们可以通过复用 QEMU 的框架来实现一个运行在用户态的虚拟机监控器。在硬件虚拟化技术出现之前，QEMU 主要通过本章介绍的软件技术（动态二进制翻译）运行虚拟机。而在硬件虚拟化技术出现之后，QEMU 也可以通过 Linux 中的 KVM 使用硬件虚拟化机制，从而大大提升运行性能。从整体框架的角度来看，QEMU 与代码片段 14.2 中的简单虚拟监控器没有太大区别，它们都需要使用 KVM 提供的 API 运行和管理虚拟机。但是 QEMU 还支持不同设备的 I/O 虚拟化，并能提供对多 vCPU 的支持以及虚拟机迁移等功能。

图 14.27 展示了 QEMU 使用 KVM 运行虚拟机的过程。运行在用户态的 QEMU 是一个进程，对应一个虚拟机。在 QEMU 进程内，每一个用户态线程负责运行虚拟机内的一个 vCPU。具体而言，一个 QEMU 用户态线程管理一个对应 vCPU 的所有寄存器状态，当此线程调用 KVM_RUN 接口时，KVM 会将对应 vCPU 的所有寄存器状态加载到硬件中并开始执行。对于 Linux 内核而言，调度该用户态线程相当于调度此 vCPU。这种方式非常巧妙地复用了 Linux 的调度机制，因而不需要为虚拟化重新实现一套新的调度算法。

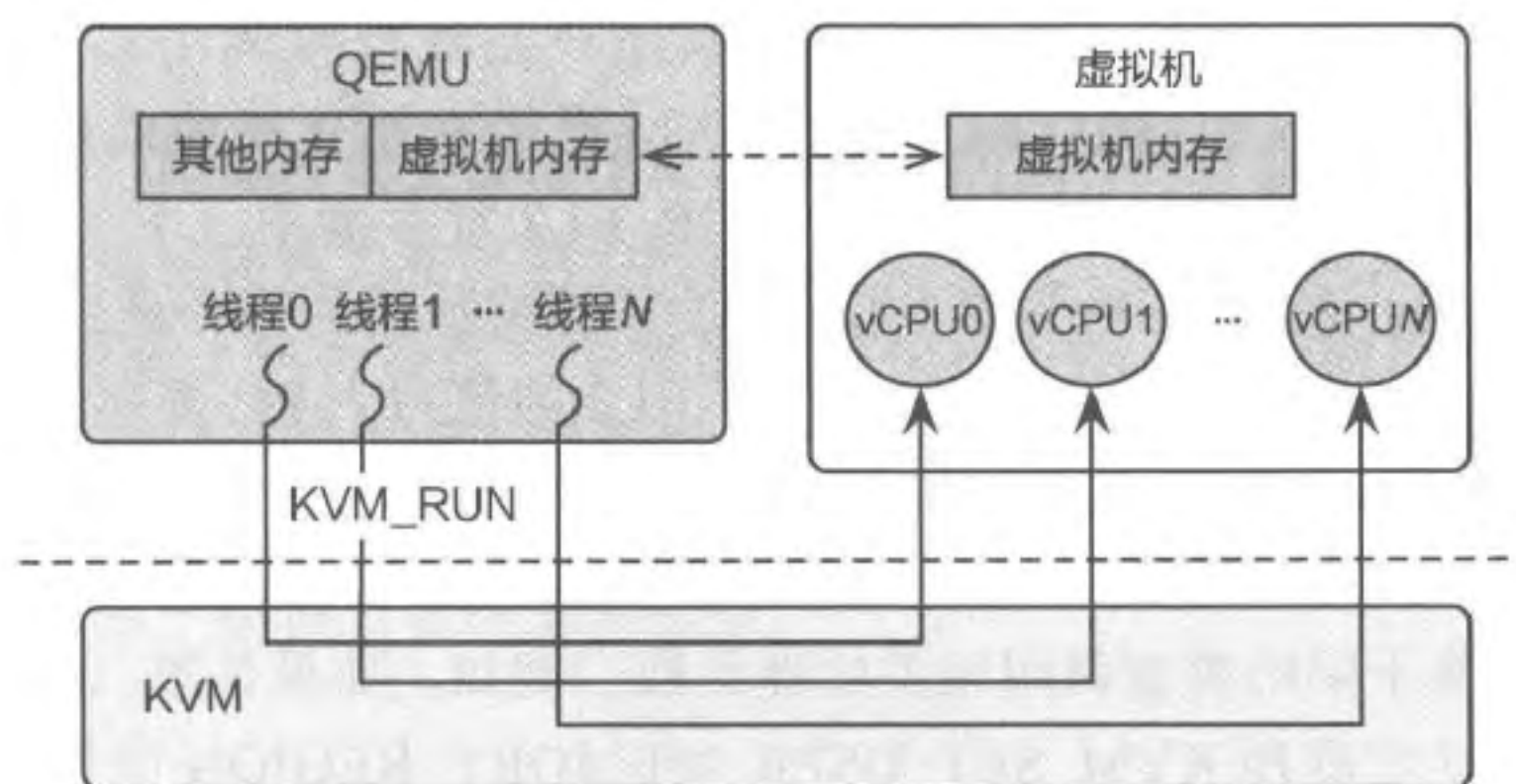


图 14.27 QEMU 和 KVM 在架构上的关系

作为一个用户态进程，QEMU 拥有自己的虚拟地址空间，但作为用户态虚拟机监控器，它还需要为虚拟机分配客户物理内存。为了支撑虚拟机的运行，QEMU 使用 mmap 系统调用分配一大段连续的内存区域，并将此区域作为虚拟机的客户物理地址空间。如代码片段 14.2 所示，QEMU 在运行虚拟机之前，使用 `ioctl(KVM_SET_USER_`

MEMORY_REGION) 系统调用将此内存区域映射给虚拟机，这段内存区域会被所有 vCPU 共享。此外，因为虚拟机内存本质上属于 QEMU 进程内部的内存，所以 QEMU 可以直接读写虚拟机的内存数据。这不仅可以加速 I/O 虚拟化的数据传输（例如 QEMU 直接读取虚拟机指定的 DMA 内存区域），也可以方便 QEMU 实现虚拟机热迁移等功能。

QEMU 同样提供对 I/O 虚拟化的支持，它既可以提供模拟的设备，也可以提供半虚拟化的设备模型。为了提升 I/O 虚拟化的性能，QEMU 同样可以使用设备直通的方式。因为本节主要介绍 QEMU 与 KVM 的协同关系，所以不再对 QEMU 的 I/O 虚拟化机制做进一步介绍。

14.7.3 KVM 内部实现简介

前文已经介绍过 KVM 实际上是硬件虚拟化机制的驱动程序。作为一个驱动程序，KVM 的实现分为两个部分。第一部分是向用户态提供 API 驱动支持，允许用户态程序（如 QEMU）管理虚拟机相关的状态。KVM 作为一个 Linux 内核驱动模块，在初始化时会在内核中注册一个字符驱动，该字符驱动支撑 /dev/kvm 设备文件的所有功能，例如 `ioctl` 等。当用户态程序使用 `ioctl` 系统调用与 KVM 设备文件交互时，KVM 根据系统调用传入的 API 调用相应的函数。由于 `ioctl` 的实现部分较为简单，并且与系统虚拟化没有直接关系，我们不对其进行更为深入的介绍，感兴趣的读者可以阅读 Linux 驱动相关的书籍。

KVM 实现的第二部分是运行虚拟机并处理虚拟机下陷，如图 14.28 所示。当 QEMU 完成虚拟机以及每个 vCPU 的初始化之后，其用户态线程将调用 `KVM_RUN` API，通知 KVM 正式执行对应 vCPU 的代码。此时 KVM 已经初始化完成此虚拟机相关的数据结构，例如将第二阶段页表基地址写入 `VTTBR_EL2` 寄存器中。下一步，KVM 设置此 vCPU 相关的寄存器，包括 `ELR_EL2`（指向 vCPU 即将执行的指令地址）和通用寄存器。当 KVM 执行 `eret` 指令时，硬件自动将 `ELR_EL2` 的值写入 `PC` 寄存器，并正式开始执行 vCPU 的代码。如果在虚拟机执行过程中发生中断或异常，将触发虚拟机下陷，CPU 从 `EL0/EL1` 切换至 `EL2`，并调用中断处理函数，最终回到 KVM。在处理虚拟机下陷时，KVM 根据此次下陷的类型调用相关处理函数。例如，如果是第二阶段页表的缺页异常，将使用 QEMU 之前用 `KVM_SET_USER_MEMORY_REGION` 设置的客户物理地址区域处理此次下陷。如果 KVM 能够成功处理此次下陷，将恢复 vCPU 的执行。图 14.28 中的 `while` 循环展现了 KVM 在执行 vCPU 时的基本逻辑。如果 KVM 无法处理此次虚拟机下陷，例如需要进行 I/O 虚拟化相关的操作或需要 QEMU 分配虚拟机内存，它将回到 QEMU 进程中，交给相关线程处理。QEMU 处理完成后，再次调用 `KVM_RUN` API 恢复 vCPU 的执行。

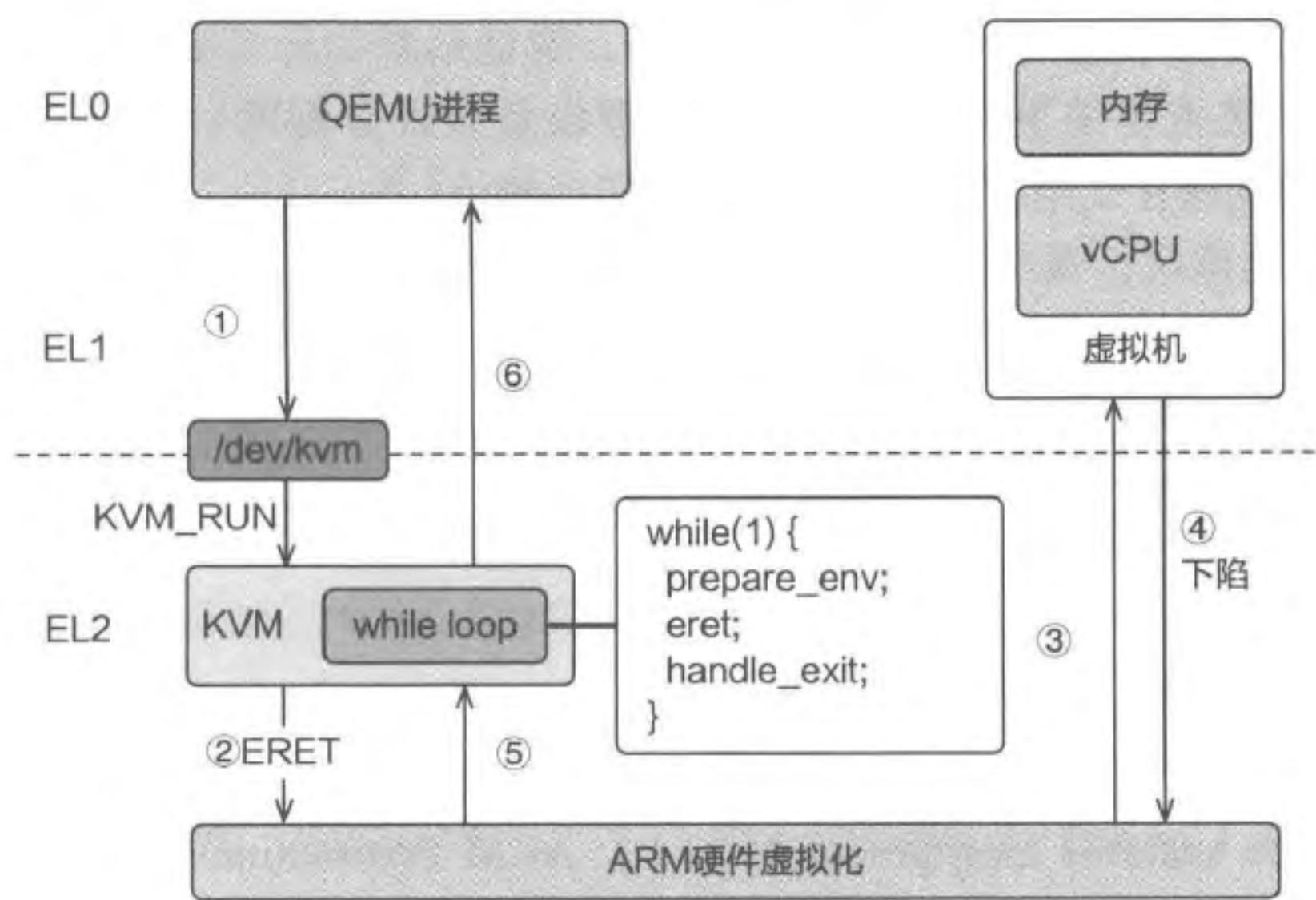


图 14.28 KVM 执行 vCPU 的基本过程

14.8 思考题

1. 假设有 10 个虚拟机，每个虚拟机内有 10 个进程，如果通过影子页表实现内存虚拟化，总共需要多少影子页表？如果通过第二阶段页表实现内存虚拟化，总共需要多少第二阶段页表？
2. 尽管第二阶段页表翻译技术在很多情况下性能优于影子页表，但在某些情况下，影子页表的性能要优于第二阶段页表技术。请提供一个示例并说明原因。
3. 假设我们使用第二阶段技术来虚拟化客户虚拟机的内存（页表均为 4 级）。请描述硬件如何将客户虚拟地址转换为相应的主机物理地址。提示：可以忽略 TLB 的影响。
4. 在 EL1 或 EL0 模式下执行特权指令将触发虚拟机下陷，将控制权转移给虚拟机监控器。在处理完虚拟机下陷之后，虚拟机监控器将控制权交还给虚拟机。以下是几种可能触发虚拟机下陷的例子：
 - 访问内存导致第二阶段页表的缺页异常。
 - 使用 wfi 指令。
 - 在使用第二阶段页表时修改 TTBR0_EL1 寄存器的值。请判断它们是否会引起虚拟机下陷。如果是，请进一步回答在虚拟机恢复执行之后，是否需要重新执行此条引起下陷的指令。
5. 在半虚拟化技术中，Virtio Virtqueue 技术相对于指令级模拟的优势体现在哪里？
6. 在 x86 和 ARM 平台引入硬件虚拟化技术之后，指令模拟和半虚拟化技术是否还有存在的意义？